Manual

Quadrupedal navigation under partial observability

Implementation guide and theory notes | October 14, 2026

For Tanmay Kumar, Amedeo Zaccone and Damien Bardina

This project combines a high-level pile planner with a learned Go1 locomotion controller. The planner chooses where to go; the controller learns how to move the robot's joints toward that target.

The implementation includes CPU MuJoCo, a CUDA MuJoCo Warp backend, PPO training, classical and learned planners, terrain generation, checkpoint export and local execution. It is ready for the team's cloud preflight and training work. It does not include a trained, validated walking policy.

Validation boundary: CPU tests and short training/export/inference runs were exercised locally. NVIDIA CUDA execution, long training, locomotion quality and research conclusions remain to be validated on the team's H100/A100 and held-out layouts.

The code uses direct MuJoCo Warp and PyTorch rather than the proposal's mjlab task framework. The implementation choices and limitations are listed in this manual and in docs/IMPLEMENTATION_STATUS.md.

Reading order

1. Architecture and information boundaries
2. Terrain, observations and actions
3. PPO actor-critic theory and implementation
4. Rewards and critical design choices
5. Classical and learned planning
6. Cloud training and local deployment
7. Evaluation, calibration and research validity
8. Important functions and troubleshooting

1. Architecture and information boundaries

At deployment, the robot follows this loop:

```
Local pile observations + final goal
                  |
           Subgoal planner
                  |
       Selected visible pile ID
                  |
       Physics goal executor
                  |
 Relative target + heading + time budget
                  |
 PPO actor + proprioception + height scan
                  |
        12 joint position targets
                  |
       PD control + MuJoCo physics
                  |
       New pose and observations
```

The high-level planner runs on arrival or after a four-second subgoal timeout. The actor runs at 50 Hz. MuJoCo integrates physics at 500 Hz, giving ten physics steps per actor action. The controller can immediately receive a new target after an intermediate arrival; final targets use a stand-still condition.

The actor and critic are separate neural networks during PPO training. The actor outputs actions. The critic estimates expected future reward and helps compute the learning signal; it is not an expert action generator. The critic is not needed for deployment.

Teacher/student terminology belongs to a different layer: the learned planner imitates a privileged full-map A* oracle. The oracle can use the entire terrain during training and reference evaluation. Greedy, optimistic and learned planners receive only local observations and their permitted memory.

There are two execution environments. The abstract environment cheaply models movement between piles and supports planner experiments. The physics environment runs the actual Go1 model and joint controller. Success in the abstract environment does not establish locomotion success.

Current sensing assumption: perfect odometry, perfect pile association and an ideal local terrain observation. The height scan is sampled from a precomputed elevation raster, as in the actual Lab 1 implementation; it is not a real camera/LiDAR pipeline.

2. Terrains and coordinate conventions

The new generator supplies twenty difficulty rows, numbered 0 to 19. Pile size decreases from 0.50 m to 0.12 m while gaps increase from 0.03 m to 0.13 m. These endpoints are hypotheses to test, not demonstrated Go1 capability. The earlier ten-level scene generator remains available for reproducing the Day-1 examples.

Family	Main challenge	Intended use
flat	Contiguous tiled support	Initial locomotion debugging
open	Regular separated piles	Core sparse-terrain skill
holes	Random missing piles	Local detours and robustness
moat	Missing band with offset crossing	Partial-observability comparison
dead_end	Pocket and blocked direct route	Memory and escape behaviour
steps	Increasing/decreasing top heights	Vertical motion control
rough	Random pile heights	Uneven support
slalom	Tall alternating wall obstacles	Route selection and collision avoidance
bridge	Narrow support corridor	Lateral precision
slanted	Tilted pile tops	Contact and attitude robustness

Scene JSON is the shared geometry source. build_mjcf() creates MuJoCo collision geometry; Terrain.from_scene() creates the planner representation. Walls are physical obstacles and their footprints are removed from navigable support. Visual overlays are not planner inputs.

Coordinates use metres, seconds and radians. The world frame is right-handed with Z upward. The local frame follows robot yaw: X forward, Y left, Z up. It does not rotate with roll or pitch. The initial odometry frame equals the world frame.

A pile record describes its top-centre position, size and optional roll/pitch. The abstract model approximates tilted support conservatively; physical contacts use the actual tilted MuJoCo geometry. Arbitrary yaw-rotated pile footprints are rejected rather than silently interpreted incorrectly.

The local planner observes pile centres within a two-metre horizontal radius, without occlusion. The action mask is computed from local support and movement limits. A target's proximity alone does not make the approach safe: intermediate gaps are checked as well.

3. Observation and action contract

The checkpoint carries OBSERVATION_SPEC. Loading checks the contract so mismatched feature order or joint order cannot silently produce motor commands.

Actor field, in order	Values	Meaning
Body angular velocity	3	Roll/pitch/yaw rates
Projected gravity	3	Body orientation relative to gravity
Joint position minus home	12	Deviation from nominal stance
Joint velocity x 0.05	12	Scaled joint rates
Previous action	12	Previous bounded action
Relative target X/Y	2	Subgoal in yaw-only robot frame
Heading error sin/cos	2	Continuous angular encoding
Remaining time fraction	1	Four-second subgoal budget
Final-goal flag	1	Whether stopping is required
Height samples	825	33 x 25 samples over 1.6 x 1.2 m

Total: 873 actor values. The critic receives the same vector plus three body-frame linear-velocity values: 876 values. Linear velocity is privileged simulation information here; it is not supplied to the deployed actor.

Scan coordinates run from X=-0.8 to 0.8 and Y=-0.6 to 0.6 m in 0.05 m increments, flattened X-major then Y-minor. Values are base height minus terrain height minus 0.30 m, clipped to [-1,1]. The underlying elevation raster uses 0.025 m cells; this introduces a documented discretization error near edges.

The stochastic actor produces 12 Gaussian latent actions during training. Applying tanh bounds them to [-1,1]. The target for each joint is home_position + action_scale * action, clamped to the joint control limits. The default action scale is 0.5 rad. Joint order is FR, FL, RR, RL, with hip, thigh and calf per leg.

PD gains default to Kp=35 and Kd=0.5, with joint damping 0.2. Gains, control timestep and action scale travel in the checkpoint. Do not tune them only on the local machine after training and expect equivalent behaviour.

Running normalization moments are frozen during each PPO rollout/update block. The exported actor includes its normalizer. Downloading only a weight matrix and omitting normalization is not a valid deployment.

4. PPO actor-critic: brief theory

Reinforcement learning improves a policy by collecting interactions and maximizing expected discounted reward. Our actor chooses joint commands; the environment produces rewards and next observations. The critic predicts the return expected from the current observation.

PPO compares an action's likelihood under the updated actor with its likelihood when the rollout was collected. Let r be that probability ratio and A the estimated advantage. The clipped objective uses the smaller of r*A and clip(r, 1-epsilon, 1+epsilon)*A. This limits incentives for excessively large policy changes within an update [1]. Our default epsilon is 0.2.

Generalized advantage estimation combines temporal-difference residuals across time. The residual is reward + gamma * next_value - current_value. GAE discounts later residuals by gamma*lambda, trading variance against bias [2]. Defaults are gamma=0.99 and lambda=0.95.

A true terminal event, such as a fall, has no continuation value. A time-limit truncation can still have future value: the trainer bootstraps from the terminal observation before the environment resets. The implementation tests this distinction explicitly. GAE recursion stops at every episode boundary, preventing leakage across resets.

The training loss combines the clipped actor loss, critic mean-squared error, entropy encouragement and reflection consistency. Gradients are clipped. An approximate KL threshold stops additional PPO epochs when the policy changes too much. These are stabilization tools, not guarantees of convergence.

Reflection consistency encourages a mirrored observation to produce mirrored joint actions. Left/right legs swap; hip signs and lateral/angular features change appropriately. This implementation uses a consistency regularizer rather than adding mirrored samples to the PPO likelihood objective.

The CUDA backend batches MuJoCo Warp worlds on the NVIDIA GPU [4]. The native backend uses CPU MuJoCo. Both share environment, reward and policy code. All worlds in a batch share terrain geometry, while initial poses, commands and actions vary. The terrain is regenerated between rollout blocks, not inside a collected trajectory.

Practical implication: a successful optimizer step proves only that the data flow and loss work. It does not prove the robot can walk. Judge behaviour using arrivals, falls, timeouts and inspected trajectories.

5. Rewards: Lab 1 categories, adapted to goals

The actual lab1_C notebook was inspected. It grouped rewards into command following, body control, foot behaviour, efficient control and failure prevention. This project retains those categories and adds subgoal arrival and final stopping. The weights are starting values in EnvConfig, not experimentally tuned claims.

Category	Implemented terms	Purpose / risk
Goal progress	Position near deadline, heading near goal, arrival bonus	Reward reaching the correct place, not just moving
Motion tracking	Desired planar velocity, desired yaw rate, early progress shaping	Bootstrap useful movement toward the target
Body stability	Tilt, vertical velocity, roll/pitch rate, nominal pose	Discourage bouncing or uncontrolled posture
Foot behaviour	Clearance, slip, swing duration	Encourage useful stepping; proximity-based contact signals are approximate
Smoothness/effort	Action change, torque, mechanical power, joint acceleration	Reduce jitter and wasteful control
Safety/stopping	Soft joint limits, stall, fall, final stand-still	Penalize collapse and enforce terminal stopping

Continuous reward terms are multiplied by the 0.02 s control timestep. Arrival and fall are discrete event rewards, implemented so their magnitude is not reduced by that timestep. Every weighted term is logged separately, making reward domination visible.

A desired planar velocity is derived from the subgoal direction and distance, capped at 0.6 m/s. This adapts Lab 1's velocity-tracking objective to goal commands. An early progress term is disabled after 150 PPO updates. Intermediate targets chain immediately; a final target rewards holding position.

Critical checks:

• A controller can exploit a progress reward by lunging and falling. Compare arrival and fall rates, and inspect physics rollouts.

• Excessive stability/energy penalties can produce standing still. Check stall, tracking and arrival terms together.

• Large clearance rewards can create exaggerated stepping. Foot-clearance reward is bounded, but still needs tuning.

• A revisiting penalty is inappropriate for planner escape: returning over old piles may be the only valid route.

• Hard terrains may contain unreachable random targets. Begin with flat/easy support, verify locomotion, then increase difficulty.

• Training a good local controller does not solve global route selection; keep the two layers' metrics separate.

6. Classical planners and memory

Greedy: choose the eligible visible pile nearest the final goal in straight-line distance. It has no memory. It may turn or backtrack, but it does not understand whether a local choice leads into a hidden pocket. It is intentionally retained as a baseline.

Full-map oracle: run A* on the complete pile graph. Nodes are piles; edges are allowed straight-line subgoal transitions. Edge costs are distance, and the heuristic is remaining Euclidean distance. A* prioritizes f=g+h. It can accept a temporary detour because it knows that route reaches the goal. It uses the same abstract movement rules as the other planners; it is not an omnipotent physical robot.

Optimistic replanner: retain observed support and gaps in an occupancy grid. Unseen cells are provisionally traversable. Reverse Dijkstra computes cost-to-go from the goal; this is the zero-heuristic case of A*. The next action is always a currently visible, eligible pile. Unknown space may influence a tentative plan but is never directly commanded as an invented foothold.

The optimistic map uses a conservative inner observation region before declaring absent support, because visibility filters pile centres. A boundary pile can extend into the sensor circle while its centre is outside. Previously observed terrain is retained, and memory resets between episodes.

The grid has finite resolution and a search budget. It is not claimed complete or physically clearance-safe. A* optimality also refers only to the constructed graph and its costs, not actual energy, stability or time-optimal Go1 motion.

Abstract execution: the original model uses deterministic point-body movement with configurable gap/height/time limits. A target 1-2 m away may cross several intermediate supports; the straight approach is checked. The calibrated wrapper can instead sample measured transition success. Neither model replaces final physics evaluation.

The blue path in the original replay is greedy's actual abstract trajectory. The dashed orange path is the oracle reference, not an instruction to greedy. Local coverage is light green with a green border. The exported JSON also contains the optimistic episode for analysis.

7. Learned planner, DAgger and memory

The learned planner scores the visible pile set. Each pile is encoded by a small MLP using relative top position, dimensions and normal. Mean/max pooling provides a scene context. The final scorer produces one logit per pile, with invalid actions masked out. Raw IDs and the full map are not neural input features.

Variant	Memory	Experiment meaning
L0	None	Can local perception alone choose useful subgoals?
L1	GRU hidden state	Does learned temporal memory help?
L2	Per-pile visit count	Does explicit visitation memory help?
L3	GRU + visit count	Do both forms complement each other?

DAgger addresses a weakness of ordinary imitation: a student visits states that may not appear in expert demonstrations. During collection, a mixture of student and oracle actions drives the simulator. The full-map oracle labels the next subgoal at visited states. New trajectories are aggregated with previous data and the scorer learns by cross-entropy [3].

The recurrent variants train on ordered episode sequences. Hidden state is reset between episodes and detached at truncated-backpropagation boundaries. Visit counts are episode-local per-pile counts in this implementation; the proposal's decaying odometry grid is not used.

Optional policy-gradient fine-tuning samples pile choices and rewards reductions in oracle walkable distance, plus successful completion. Oracle distance is a privileged training reward, never a deployment input. This initial optional stage uses REINFORCE; it is not the locomotion PPO loop.

A student may still fail to imitate perfectly. Two identical local views can require different actions because the oracle knows hidden terrain. Memory can help after exploration, but cannot reveal facts never observed. Report this information gap rather than describing all imitation errors as an optimizer bug.

Training seeds stay below 100,000; generated validation/test layouts use separate ranges. Holdout scenes must remain separate when selecting checkpoints. A planner trained in the deterministic surrogate must be tested again with the frozen controller in physics.

8. H100/A100 cloud training

Use Linux with a working NVIDIA driver and persistent storage. Clone the private repository using normal GitHub access, select the reviewed tanmay branch, then run:

```
uv sync --extra cloud --frozen
bash scripts/cloud_setup.sh
```

The setup compares native and Warp standing dynamics and runs two small GPU PPO updates. Stop on CUDA errors, contact/constraint overflow, nonfinite values or excessive parity error. This Mac cannot certify those GPU checks. Reduce parallel world count if memory is insufficient; increase capacity only when the preflight identifies a contact/constraint allocation problem.

For the main run:

```
uv run --extra cloud quad-train --config configs/cloud.json
```

The default requests 131,072,000 transitions. It starts on easy open piles and cycles open, holes, rough, steps and slanted families. To establish walking first, create a flat-only config with the same architecture and control contract, then warm-start the obstacle run:

```
uv run --extra cloud quad-train --config configs/cloud.json \
  --warm-start runs/flat/last.pt
```

To continue an interrupted run:

```
uv run --extra cloud quad-train --config configs/cloud.json \
  --resume runs/open-seed0/last.pt
```

A resume restores policy, optimizer, normalization and update count, but resets simulator state. It is not bit-identical reproduction of uninterrupted GPU execution. Do not reuse an output directory for an unrelated run.

Check metrics.jsonl for each reward term, arrivals, falls, timeouts and samples/second. Checkpoints are written atomically every 50 updates. Long jobs should run in the cloud provider's persistent job/session mechanism so closing your laptop does not terminate them.

RQ1 configs in configs/experiments compare smooth versus uniform rows for seeds 0,1,2 under equal update budgets. The implemented curriculum is global per batch, not per robot. Record this difference and actual GPU throughput/memory in the report.

9. Download and local execution

Choose a checkpoint using held-out behaviour. High reward alone is not sufficient. Export on the cloud:

```
uv run --extra cloud quad-train \
  --export runs/open-seed0/last.pt \
  --bundle artifacts/controller
```

The resulting ZIP contains controller.pt, actor.ts and manifest.json. The TorchScript actor includes normalization and tanh output. The manifest records feature/joint order, control configuration, versions, source revision and file hashes. The repository supplies robot meshes and runtime code.

Download from your LOCAL terminal, substituting your provider's actual SSH user, host and checkout path:

```
scp user@GPU_HOST:/workspace/project/artifacts/controller.zip \
  ./artifacts/
unzip artifacts/controller.zip -d artifacts/controller
uv sync --extra train --frozen
```

Start the native viewer:

```
uv run --extra train python play.py \
  --checkpoint artifacts/controller/controller.pt \
  --planner optimistic --family open --row 0
```

On macOS, play.py uses mjpython. For headless evaluation:

```
uv run --extra train quad-evaluate \
  --checkpoint artifacts/controller/controller.pt \
  --planner optimistic --family dead_end --row 0
```

The physics executor obtains the actual continuous robot pose, constructs a local planner observation, validates the selected target and updates the controller command. It replans on arrival or a four-second timeout. A fall ends the episode. Final success requires staying near the final goal with plausible support height for half a second; it is a simulation criterion, not a physical safety certificate.

Do not change observation order, scan dimensions, normalization, joint order, action scale or PD gains between training and inference. CPU and GPU solvers can differ; compare paired rollouts after download. This workflow runs the Go1 model locally in simulation. It does not command real hardware.

10. Evaluation and calibration

Create a saved benchmark manifest so every planner receives identical layouts:

```
uv run --extra train quad-benchmark --create \
  --count 20 --split test --row 5 \
  --manifest benchmarks/test-row5.json
uv run --extra train quad-benchmark \
  --manifest benchmarks/test-row5.json \
  --output outputs/benchmarks/abstract
```

Add --checkpoint artifacts/controller/controller.pt to run the same list in physics. Add --planners learned and --planner-checkpoint PATH to evaluate a learned planner. Use the same frozen controller checkpoint for all planner comparisons.

The manifest stores the actual scene JSON, seed and pocket depth, rather than relying only on a generator version. Output includes individual trajectories and a summary with success fractions and Wilson 95% intervals. Failed short paths are not reported as efficient successful routes. Pocket entry/exit is measured against the generated rectangular pocket, sampling path segments at 5 cm. Escape requires exit and eventual success. scripts/summarize_runs.py reports mean/std across independent training runs.

The abstract oracle is an upper bound only within its movement model. If oracle physics success is poor, investigate controller reachability and executor assumptions before blaming local planning. Keep impossible-layout counts explicit; do not remove inconvenient failures after seeing results.

Measure short-goal success after freezing a controller:

```
uv run --extra train python -m quadruped_nav.calibrate \
  --checkpoint artifacts/controller/controller.pt --trials 20
```

Calibration currently groups trials by family, difficulty row and target length. The table includes counts and empirical probabilities. The calibrated abstract wrapper refuses missing bins instead of inventing probabilities. It is coarser than a full gap/height/slant model, and few trials produce uncertain estimates. Restrict calibrated benchmarks to measured conditions or collect more data.

For defensible conclusions, separate three things: software correctness, learned locomotion quality, and evidence supporting RQ1/RQ2/RQ3. Unit tests support the first. Held-out physics runs support the second and third. Neither a successful smoke optimizer update nor a smooth-looking abstract path proves the research hypotheses.

11. Important functions

All paths below are relative to src/quadruped_nav/ unless stated otherwise.

Function / class	Location	Responsibility
generate_terrain()	terrain.py	Reproducible twenty-row terrain families
build_mjcf()	mjcf.py	Shared JSON to physical MuJoCo model
Terrain.from_scene()	abstract.py	Geometry adapter and opaque pile IDs
straight_approach()	abstract.py	Check intermediate support along a target segment
observe_pose()	abstract.py	Local observation at a continuous physics pose
AbstractEnv.step()	abstract.py	Validated abstract transition and episode log
OraclePlanner.route()	planners.py	Full-map A* reference route
OptimisticPlanner.select_subgoal()	planners.py	Update memory and choose a visible target
PileNetwork.forward()	learned.py	Masked per-pile scoring with optional GRU
train_planner()	learned.py	DAgger aggregation and optional REINFORCE
HeightMap.sample()	rl/env.py	Ideal local elevation samples
GoalEnv.observations()	rl/env.py	Fixed actor/critic feature ordering
GoalEnv.step()	rl/env.py	Joint control, rewards, termination and chaining
NativeBackend / WarpBackend	rl/backend.py	Shared CPU/GPU physics state interface
ActorCritic / RunningNorm	rl/policy.py	Networks and observation normalization
advantages() / train()	train.py	GAE, PPO updates and checkpoints
export_bundle()	rl/policy.py	Portable actor and complete checkpoint ZIP
evaluate()	evaluate.py	Planner-to-controller physics execution
benchmark()	benchmark.py	Paired experiment list and summaries
calibrate() / CalibratedEnv	calibrate.py	Empirical hop success and sampled abstract outcomes

Entry points: quad-train, quad-evaluate, quad-benchmark and quad-learn-planner. simulate.py opens the abstract replay; play.py opens local trained-policy physics. Configuration lives in configs/. Generated runs and model bundles are ignored by Git.

12. Troubleshooting and known boundaries

Symptom	First checks
CUDA unavailable	NVIDIA driver, torch CUDA build, device string, cloud instance type
Contact capacity exceeded	Increase nconmax/njmax, then repeat preflight before training
Nonfinite loss or physics warnings	Stop; inspect timestep, action scale, joint limits and reward magnitudes
Robot stands still	Arrival/stall/tracking breakdown; easy flat terrain; excessive penalties
Robot moves but falls	Action scale, tilt/foot/effort terms; curriculum too hard; unreachable targets
Checkpoint will not load	Observation/joint contract, architecture and package versions
Local behaviour differs	Normalizer, action/PD settings, scan ordering and native/Warp parity
Greedy loops	Expected baseline failure; compare optimistic memory and oracle
Learned planner plateaus	Teacher information gap, action masks, sequence memory and held-out layouts
Viewer fails on macOS	Use play.py with uv, which invokes mjpython

The implementation deliberately stops on malformed target selections, incompatible checkpoints and missing calibration bins. These errors should be investigated rather than bypassed to make a demonstration look successful.

Scope differences are explicit: direct Warp replaces mjlab; elevation-raster sensing replaces runtime ray casting; curriculum is global per batch; symmetry is a consistency loss; visit memory is per pile; calibration is coarse. The original proposal remains unchanged, and docs/IMPLEMENTATION_STATUS.md records the actual implementation.

The remaining project work is empirical: run GPU preflight, train, tune from evidence, freeze the controller, calibrate, run paired held-out benchmarks and write the findings. The code cannot promise successful locomotion before those steps.

Sources and project references

[1] Schulman et al., Proximal Policy Optimization Algorithms (2017): https://arxiv.org/abs/1707.06347

[2] Schulman et al., High-Dimensional Continuous Control Using Generalized Advantage Estimation (2015): https://arxiv.org/abs/1506.02438

[3] Ross, Gordon and Bagnell, A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (2011): https://proceedings.mlr.press/v15/ross11a.html

[4] MuJoCo Warp documentation: https://mujoco.readthedocs.io/en/latest/mjwarp/

Project sources: Project_Proposal_v2.pdf; Two-week plan subgoal planning.md; the local eai2026_lab1_rl/lab1_C.ipynb reward configuration and perceptive_go1.py. The lab's disabled optional reward terms were not blindly copied as active project objectives.